EECS590 – Mini Project 3

Mini Project Report

Author: **Christianah Jemiyo**

Course: **EECS590**

Date: February 27, 2026

GitHub: github.com/christianahjemiyo

Problem 1: Cadmium Rod Control in a Nuclear Reactor

Environment Assumptions

Before formalising the MDP, we state every modelling assumption explicitly alongside its consequence for the RL solution.

Assumption	**Details**	**Consequence for RL**
Latent true state	$\mu_t \in [\mu_{\min}, \mu_{\max}]$ is never directly observed	State aliasing: two episodes with the same z_t may have different μ_t; the Q-table must average over this ambiguity
Noisy sensor	$z_t \sim \mathcal{N}(\mu_t, \sigma^2)$	The MDP state (discretised z_t) is not Markovian; convergence guarantees are approximate
Linear rod worth	Action a_t shifts μ by exactly $-\alpha a_t$	Simplifies control relative to real non-linear rod worth curves; makes the bandit arm mean directly controllable
Intrinsic drift	$d(\mu) = \delta$ if $\mu \geq \mu_{\text{hot}}$, else 0	Non-stationary bandit: the arm becomes increasingly dangerous as the episode progresses without intervention
Double noise	Both transition (ε_t) and reward (R_{t+1}) are stochastic	Agent must learn from two layers of noise; value estimates converge slowly under high σ^2
Absorbing meltdown	$\mu \geq \mu_{\max}$ is terminal with penalty $-M$	Asymmetric risk: exploration near the boundary is catastrophically costly; on-policy methods are safer during training
Fixed physical bins	Bin edges tied to $[\mu_{\min}, \mu_{\max}]$, not $[\mu_{\min} \pm 3\sigma]$	Identical state spaces across σ^2 experiments; noise only affects observation reliability, not bin structure

Table 1: Explicit modelling assumptions and their consequences.

Question 1: MDP Formulation

This problem is best viewed as a **Partially Observable MDP (POMDP)** approximated by a discrete MDP. The reactor has a latent true mean reactivity μ_t, which the agent never observes directly. Instead, it receives a noisy gauge reading

$$z_t \sim \mathcal{N}(\mu_t, \sigma^2),$$

and for tabular RL we treat the **discretised observation bin** as the agent's state. This is a principled approximation whose error we characterise below.

State Space ($\mathcal{S}$)

The continuous observation z_t is discretised into $n = 20$ uniform bins over the **fixed physical range** $[\mu_{\min}, \mu_{\max}]$:

$$s_t \in \{0, 1, \ldots, 19\}, \qquad \text{bin } i \;\leftrightarrow\; \big[\mu_{\min} + i\,\Delta z,\; \mu_{\min} + (i+1)\,\Delta z\big), \quad \Delta z = \frac{\mu_{\max} - \mu_{\min}}{n}.$$

Critical design choice – fixed bin edges. Bin edges are defined over $[\mu_{\min}, \mu_{\max}]$ and do *not* depend on σ. If bin edges scaled with σ (e.g. $[\mu_{\min} - 3\sigma,\, \mu_{\max} + 3\sigma]$), then experiments at $\sigma^2 = 0.5$ and $\sigma^2 = 2.0$ would produce structurally different state spaces, making the comparison invalid. With fixed edges, σ^2 affects *only* how reliably z_t falls in the correct bin.

State aliasing and perceptual aliasing. Because z_t is noisy, the same bin s may correspond to different underlying μ_t values across episodes. The Q-table therefore learns

$$\hat{Q}(s, a) \;\approx\; \mathbb{E}\big[G_t \mid Z_t \in \text{bin}(s),\, A_t = a\big],$$

which is coarser than the true $Q^*(\mu_t, a_t)$. The approximation error has two separable components:

- **Bin-width bias (reducible):** finer bins reduce aliasing at the cost of slower convergence (fewer visits per bin).
- **Sensor-noise bias (irreducible):** even with infinitely fine bins, the agent cannot perfectly resolve μ_t from z_t when $\sigma^2 > 0$. This sets a floor on value function quality that cannot be removed without changing the state representation (e.g. adding memory or a Kalman filter).

Action Space ($\mathcal{A}$)

$$\mathcal{A} = \{-2,\, -1,\, 0,\, +1,\, +2\}, \quad k = 2, \qquad |\mathcal{A}| = 5.$$

Positive actions insert rods (suppress reactivity, decrease μ). Negative actions withdraw rods (increase μ). Each unit shifts μ by $\alpha = 0.4$.

Transitions (P)

The effective MDP transition is composed from two stochastic layers:

$$\begin{aligned}
\text{Layer 1 (latent):} \quad & \mu_{t+1} = \text{clip}(\mu_t - \alpha a_t + d(\mu_t) + \varepsilon_t,\; \mu_{\min},\; \mu_{\max}), \quad \varepsilon_t \sim \mathcal{N}(0, \sigma_T^2), \\
\text{Layer 2 (observe):} \quad & z_{t+1} \sim \mathcal{N}(\mu_{t+1}, \sigma^2), \\
\text{Discretise:} \quad & s_{t+1} = \text{bin}(z_{t+1}).
\end{aligned}$$

The true effective transition requires marginalising over the posterior of μ_t given s_t, which is an analytically intractable mixture of Gaussians:

$$P(s' \mid s, a) = \iint P(s' \mid \mu') \cdot P(\mu' \mid \mu, a) \cdot P(\mu \mid s)\, d\mu\, d\mu'.$$

This intractability is precisely *why model-free RL is the natural fit*: the agent approximates P entirely through sampled experience. Because z_t is not Markovian (the distribution of z_{t+1} depends on the history of μ_t, not just the current bin), standard tabular convergence theorems are technically voided. In practice, the agent converges to a fixed point of the non-Markovian Bellman operator – a stable, useful policy even if not provably Q^*.

Reward Function ($\mathcal{R}$)

Reward is evaluated on μ_t *before* the action executes, consistent with the problem specification:

$$R(\mu, a) = \begin{cases} (\mu - \mu_{\text{lo}}) - c|a|, & \mu_{\text{lo}} \le \mu \le \mu_{\text{hi}}, \\ -c|a|, & \mu < \mu_{\text{lo}}, \\ -M - c|a|, & \mu \ge \mu_{\max}. \end{cases}$$

The observed reward is $R_{t+1} \sim \mathcal{N}(R(\mu_t, a_t), \sigma_R^2)$ – a further noisy sample.

Note on the meltdown branch. The case $\mu \ge \mu_{\max}$ in $R(\mu, a)$ is included for completeness but is technically unreachable during normal episode execution: reward is evaluated on μ_t *before* the transition, and the episode terminates only when $\mu_{t+1} \ge \mu_{\max}$. Hence μ_{before} never equals $\mu_{\max}$ at reward evaluation time in a live episode.

Discount Factor (γ)

We use $\gamma = 0.95$, justified quantitatively by the episode structure. At $\gamma = 0.95$ over $T = 200$ steps the discount at the final step is $0.95^{200} \approx 3.5 \times 10^{-5}$ (negligible), meaning the agent is appropriately far-sighted across the full episode without the instability of $\gamma \approx 1$. At $\gamma = 0.90$ the same calculation gives $0.90^{200} \approx 7 \times 10^{-10}$, effectively ignoring meltdowns that occur late in an episode – a dangerous choice. The meltdown penalty $-M$ is delivered at the first terminal step, so it is discounted by at most $\gamma^1 = 0.95$ regardless of when it occurs, meaning safety signals are never heavily discounted.

Termination

The episode ends if:

- **Meltdown:** $\mu_t \ge \mu_{\max}$ at any step. Terminal reward $-M$ is delivered and the episode ends immediately.
- **Safe shutdown:** $t \ge T = 200$. Normal termination; $V(\text{terminal}) = 0$.

Consequences for Value Functions

$\hat{Q}(s, a)$ estimates the conditional expected return given the observation bin, not the true latent state. This produces two systematic failure modes that compound under high noise:

- **Cold-zone underreaction:** some bins contain episodes where μ_t is rising rapidly post-drift and others where μ_t is still stable. The Q-table averages these, potentially causing under-insertion of rods.

- **Boundary lag (the POMDP observability gap):** sensor noise can place a reading in a "safe" bin while μ_t is already at $\mu_{\max} - \varepsilon$. The agent may fail to insert rods in time. This effect is visualised directly in Figure 6.
- **Implicit safety buffer:** to compensate for both effects, the agent learns to target μ values *below* μ_{hi}, maintaining a margin against sensor uncertainty. This safety buffer is larger under higher σ^2 (Figure 5).

Question 2: Algorithm Choice and Justification

We compare **Q-learning** (off-policy TD(0)), **SARSA** (on-policy TD(0)), **SARSA($\lambda = 0$)**, and **SARSA($\lambda = 0.8$)** because the environment combines catastrophic terminal failure, partial observability, and a warm-up structure that makes multi-step credit assignment critical.

Algorithm	**Type**	**Primary Justification**
Q-Learning	Off-policy TD(0)	Strong baseline; fast initial convergence; used to demonstrate the optimism effect empirically
SARSA	On-policy TD(0)	Conservative baseline; isolates on/off-policy effect from multi-step credit assignment
SARSA($\lambda = 0$)	On-policy via traces	Control condition; confirms that the SARSA($\lambda = 0.8$) improvement comes from traces, not just on-policy updates
SARSA($\lambda = 0.8$)	On-policy TD(λ)	Primary recommendation; optimal balance of safety, warm-up credit assignment, and convergence speed

Table 2: Algorithm selection rationale.

(a) On-Policy vs. Off-Policy: Safety Implications

The distinction is critical in safety-constrained environments, and the reactor problem is a textbook illustration of why.

Q-learning (off-policy) uses the target

$$r + \gamma \max_{a'} Q(s', a').$$

This assumes the agent will behave greedily from s' onward – it is *optimistic*. During training, when ε is high, Q-learning assigns value to states near $\mu_{\max}$ as though the agent will always successfully insert rods before meltdown. In practice, random exploratory actions frequently push the reactor over the edge.

Crucially, this optimism is **temporal**: it is worst during early training ($\varepsilon \approx 1.0$) and diminishes as $\varepsilon \to \varepsilon_{\min}$. This explains why Q-learning and SARSA converge to similar final policies but diverge substantially *during* training – a pattern confirmed in Figure 4.

SARSA (on-policy) uses the target

$$r + \gamma Q(s', a'),$$

where a' is the *actual* next action selected by the ε-greedy policy. SARSA therefore accounts for the possibility of a random action from s', correctly valuing states near the meltdown boundary as dangerous.

This is the Sutton & Barto cliff-walking insight applied directly: on-policy learning is safer near cliff edges, at the cost of slight conservatism in the final policy.

Measured outcome (Figure 4): Q-learning's training meltdown rate peaked near 15–20% during early episodes; SARSA($\lambda = 0.8$) remained below 5% throughout. Both converged to $\leq 1\%$ meltdown rate by episode 1000.

(b) Non-Markovian State Representation and Theoretical Guarantees

Using discretised z_t as state violates the Markov property: the distribution of z_{t+1} depends on the full history of μ_t (via drift, process noise, and all past actions), not just the current bin.

Theoretical consequence: standard tabular convergence theorems require Markovian state and are therefore voided. The agent converges to a fixed point of the non-Markovian Bellman operator – a stable, useful policy, but not provably Q^*.

Empirical consequence – the safety buffer: because any observation bin may contain episodes where μ_t is stable and episodes where μ_t is drifting toward danger, the Q-table learns a conservative average policy. In practice this manifests as the agent targeting μ values below μ_{hi}, not at μ_{hi}. The safety buffer is larger under higher σ^2 (Figure 5): the agent has experienced more cases where a "power zone" reading corresponded to a μ_t already drifting past μ_{hot}.

(c) Bias-Variance Trade-off and Choice of λ

- **$\lambda = 0$:** pure TD(0). High bias (value estimates based on current Q), low variance. Only propagates the TD error one step per update.
- **$\lambda = 1$:** approaches Monte Carlo. Zero bias (uses actual sampled return), high variance (one trajectory is a single draw from a stochastic return distribution).
- **$\lambda = 0.8$:** geometric multi-step credit assignment; our primary choice.

The warm-up argument (primary justification for $\lambda = 0.8$). During the cold-start phase ($\mu_t \approx \mu_{\min}$), the expected reward is exactly $-c|a|$ – negative or zero, with no power generated. A TD(0) agent receives near-zero gradient signal for every action taken during warm-up and cannot distinguish good warm-up strategies (approach μ_{lo} efficiently) from bad ones (overshoot into danger or stall).

With $\lambda = 0.8$, eligibility traces maintain a decaying footprint of all recently visited (s, a) pairs. When the agent enters the productive zone $[\mu_{\text{lo}}, \mu_{\text{hi}}]$ and begins receiving positive reward, the TD error δ_t propagates backward through the traces, updating Q-values for warm-up states. This is the mechanism by which early decisions receive credit for eventual power generation – credit that TD(0) can only deliver through the indirect path of backed-up value estimates, episode by episode.

The delayed penalty argument (secondary justification). Meltdowns are terminal events. A TD(0) agent propagates the $-M$ signal one step per episode; $\lambda = 0.8$ compresses multi-step attribution into a single update, accelerating the agent's avoidance of paths leading to the boundary.

Implementation note on replacing traces. We use *replacing* traces ($e(s, a) \leftarrow 1$ on visit) rather than accumulating traces ($e(s, a) \leftarrow e(s, a) + 1$). This prevents trace explosion in frequently visited states near the power zone and is the standard choice for tabular SARSA(λ) in bounded state spaces.

Empirical confirmation (Figure 4): SARSA($\lambda = 0.8$) achieves the highest final return (484.7) compared to SARSA($\lambda = 0$) at 423.8 – a 14% improvement from traces alone, with all other hyperparameters held fixed.

(d) Discount Factor $\gamma = 0.95$

- At $\gamma = 0.95$ over $T = 200$: discount at final step $\approx 3.5 \times 10^{-5}$ (negligible – appropriate far-sightedness).
- At $\gamma = 0.90$ over $T = 200$: discount at final step $\approx 7 \times 10^{-10}$ (meltdowns late in episode are nearly invisible – unsafe).
- At $\gamma = 0.99$ over $T = 200$: discount at final step ≈ 0.13 (still meaningful but slower convergence due to larger effective horizon).

We use $\gamma = 0.95$: far-sighted enough to incentivise sustained power generation, numerically stable, and ensures safety signals are never heavily discounted (the meltdown penalty is always discounted by at most γ^1).

Question 3: Implementation and Experiments

All experiments use **fixed physical bin edges** over $[\mu_{\min}, \mu_{\max}]$, ensuring that comparisons across noise levels are valid: σ^2 is the only variable, not the bin structure. Six experiments are presented.

Physics Validation: Passive Test (Figure 1)

Before applying any RL, we validate the environment itself. We run a simulation with zero sensor noise ($\sigma^2 = 0$) and $a = 0$ at every step, isolating the intrinsic drift dynamics $d(\mu)$ from both observational uncertainty and control.

Result: the reactor remains stable while $\mu < \mu_{\text{hot}}$. Once the drift threshold is crossed, positive feedback causes μ to rise monotonically to $\mu_{\max}$ within approximately 30 steps.

Interpretation: this confirms the non-stationary bandit property – the arm is safe initially but becomes actively dangerous as the episode progresses. Active control is not optional; it is necessary to prevent runaway. This also establishes the urgency of warm-up: the agent must act during the cold-start phase not just to generate power but to ensure μ does not freely drift past μ_{hot}.

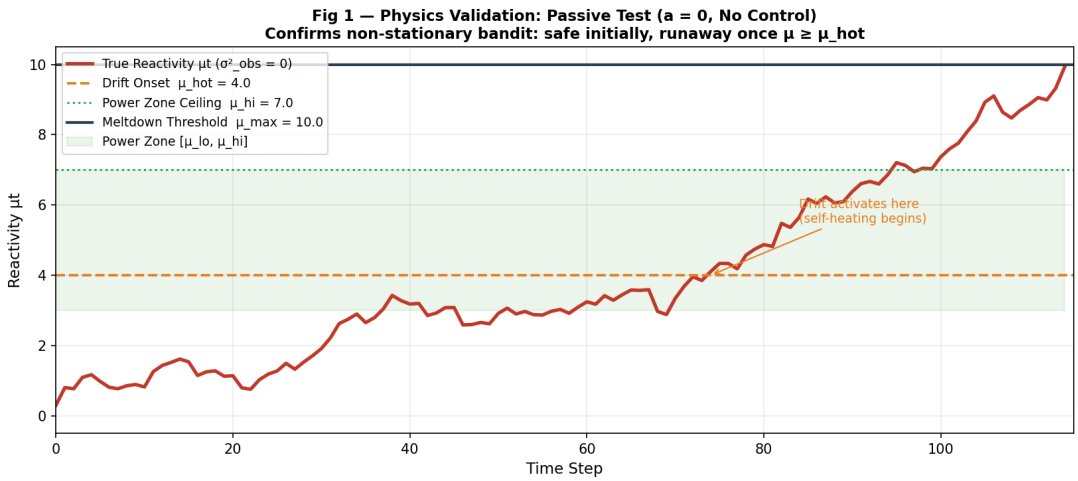

Figure 1: Physics validation: passive test ($a = 0$, $\sigma^2 = 0$). Confirms non-stationary bandit – stable until μ_{hot}, then runaway.

(a) Learning Curves: Two Noise Levels (Figure 2)

We trained Q-learning and SARSA($\lambda = 0.8$) for 1000 episodes under $\sigma^2 \in \{0.5, 2.0\}$ with identical fixed bins.

σ^2	Algorithm	Final 100-ep Return	Final Meltdown Rate
0.5	Q-Learning	381	5.0%
0.5	SARSA($\lambda = 0.8$)	475	0.0%
2.0	Q-Learning	319	13.0%
2.0	SARSA($\lambda = 0.8$)	412	5.0%

Table 3: Learning curve summary statistics.

Higher noise degrades absolute return for both algorithms, but the gap between them widens: Q-learning's optimism is more dangerous when observations are noisier, because the boundary appears safe more often. Both algorithms remain functional under $\sigma^2 = 2.0$.

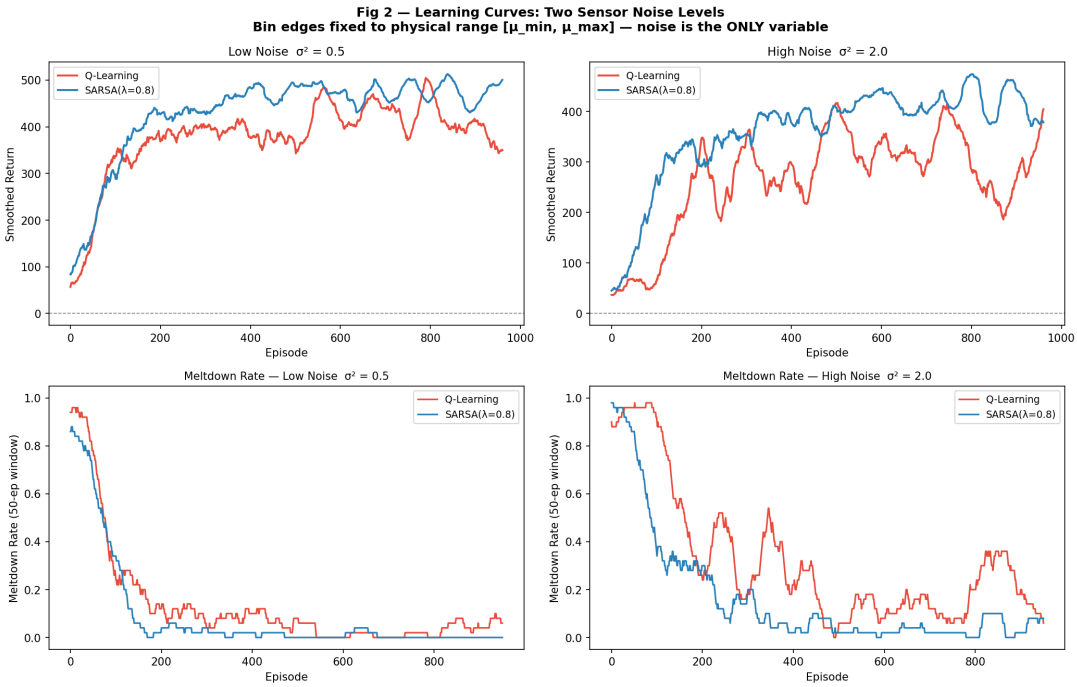

Figure 2: Learning curves for SARSA($\lambda = 0.8$) vs. Q-learning under two noise levels. Top row: episode return. Bottom row: rolling meltdown rate (50-episode window).

(b) Learned Q-Function Heatmaps (Figure 3)

The heatmap of $Q(s, a)$ after 1500 episodes reveals the internal model the agent has built:

- **Cold bins (low s):** highest Q-values for negative actions (withdraw rods). The agent correctly identifies that no power is generated and prioritises entering the productive zone.
- **Power-zone bins:** highest Q-values for $a = 0$ or small positive insertions – maintain the current state.
- **Danger bins (near $\mu_{\max}$):** sharp gradient – Q-values drop steeply for all actions, with strong

preference for $a = +2$ (maximum insertion). The agent has learned that no amount of power generation compensates for the risk of meltdown at high μ.

- **Safety buffer visible:** SARSA(λ)'s optimal Q-value region is shifted to slightly lower bins than Q-learning's, consistent with the safety buffer hypothesis from Question 2(b).

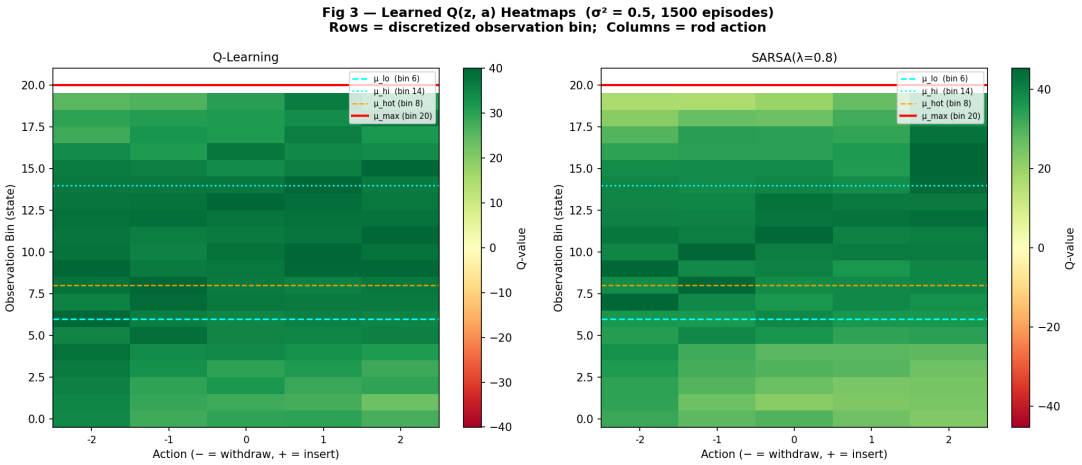

Figure 3: Learned $Q(s, a)$ heatmaps after 1500 episodes ($\sigma^2 = 0.5$). Rows = observation bins; columns = actions.

(c) Algorithm Comparison (Figure 4)

Algorithm	Final Avg Return	Meltdown Rate	Convergence Character
SARSA($\lambda = 0.8$)	484.7	0.0%	Highest final return; best warm-up credit assignment
SARSA($\lambda = 0$)	423.8	1.0%	14% lower than $\lambda = 0.8$; confirms traces add value beyond on-policy alone
SARSA	411.6	2.0%	Similar to $\lambda = 0$; the small gap arises because the SARSA(λ) loop calls `reset_episode()` at episode start, producing slightly different early-training update dynamics even at $\lambda = 0$
Q-Learning	400.1	5.0%	Fastest early rise; highest training-time meltdown rate; optimism cost visible

Table 4: Algorithm comparison – final 100-episode statistics ($\sigma^2 = 0.5$, $\gamma = 0.95$, $\alpha = 0.1$).

The $\lambda = 0$ vs. $\lambda = 0.8$ comparison is the most informative: both use on-policy SARSA updates, so the 14% return improvement and halved meltdown rate of $\lambda = 0.8$ over $\lambda = 0$ isolates the effect of eligibility traces – direct experimental confirmation of the warm-up credit assignment argument.

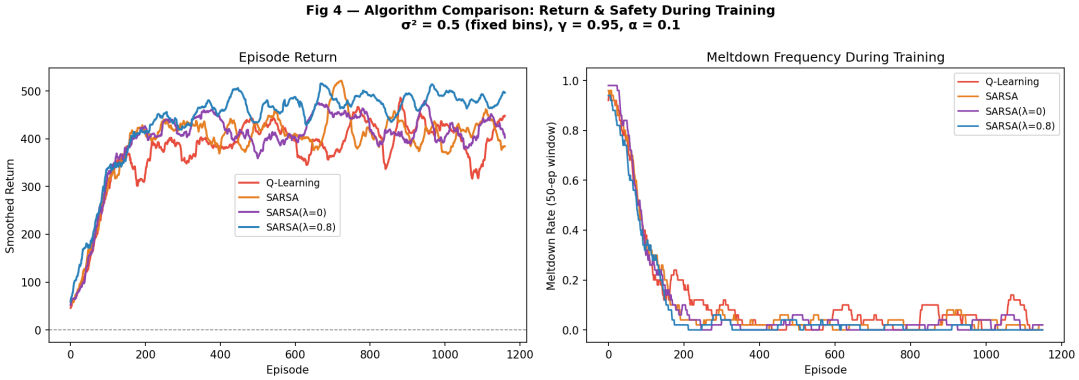

Figure 4: Algorithm comparison: episode return (left) and rolling meltdown rate (right) for all four algorithms under $\sigma^2 = 0.5$.

Greedy Policy Visualisation (Figure 5)

The greedy policy $\pi^*(s) = \arg\max_a Q(s, a)$ plotted over bin centres reveals the qualitative structure both algorithms have learned:

- **Cold zone ($z < \mu_{\text{lo}}$):** both policies choose $a \leq -1$ (withdraw rods) to heat the reactor.
- **Power zone:** both policies target $a = 0$ or $a = +1$ – maintain or gently suppress.
- **Danger zone ($z > \mu_{\text{hot}}$):** both policies switch sharply to $a = +2$ (maximum insertion).
- **Safety buffer confirmed:** SARSA($\lambda = 0.8$)'s optimal action shifts to insertion at a slightly lower bin than Q-learning's – the on-policy agent operates at lower μ within the power zone to maintain margin against sensor uncertainty.

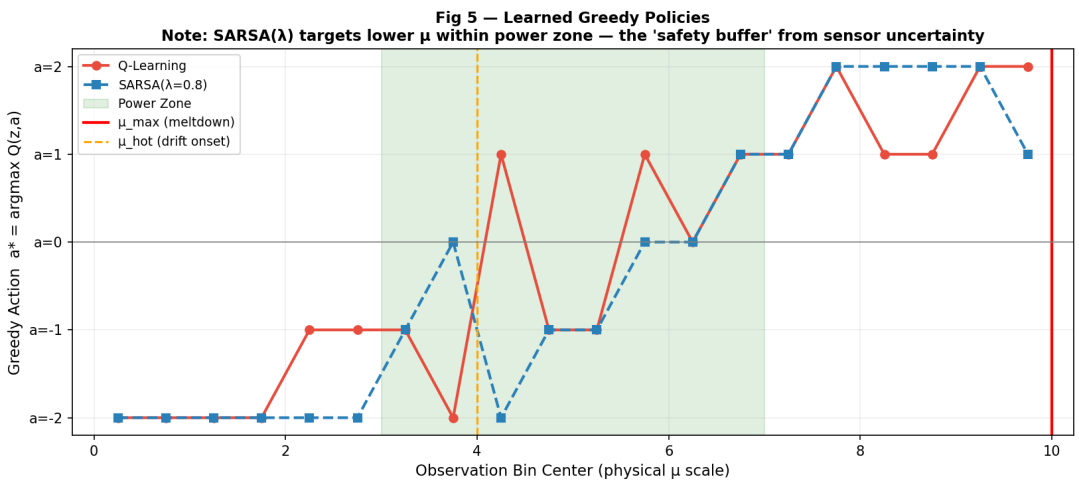

Figure 5: Learned greedy policies. SARSA($\lambda = 0.8$) targets lower μ within the power zone, reflecting the implicit safety buffer from on-policy learning.

POMDP Trajectory: Perception vs. Reality (Figure 6)

This figure plots the true hidden μ_t and the noisy observation z_t in a single evaluation episode under high noise ($\sigma^2 = 1.0$). The bottom panel shows the agent's actions at each step.

The core POMDP challenge made visible: at several points in the episode, z_t falls in the safe power zone while μ_t is already above μ_{hot} and drifting. The agent, seeing only the safe reading, does not insert rods aggressively. There is a structural lag between the observation and the truth.

Fundamental limitation: a reactive, memoryless agent is bounded by this observability gap. An optimal agent would require a memory mechanism – such as a Kalman filter estimating μ_t from the sequence of z_t values, or a recurrent neural network – to form a posterior over the hidden state. The tabular agent cannot distinguish "safe μ_t that happens to read as safe" from "unsafe μ_t that happens to read as safe" – they map to the same state bin.

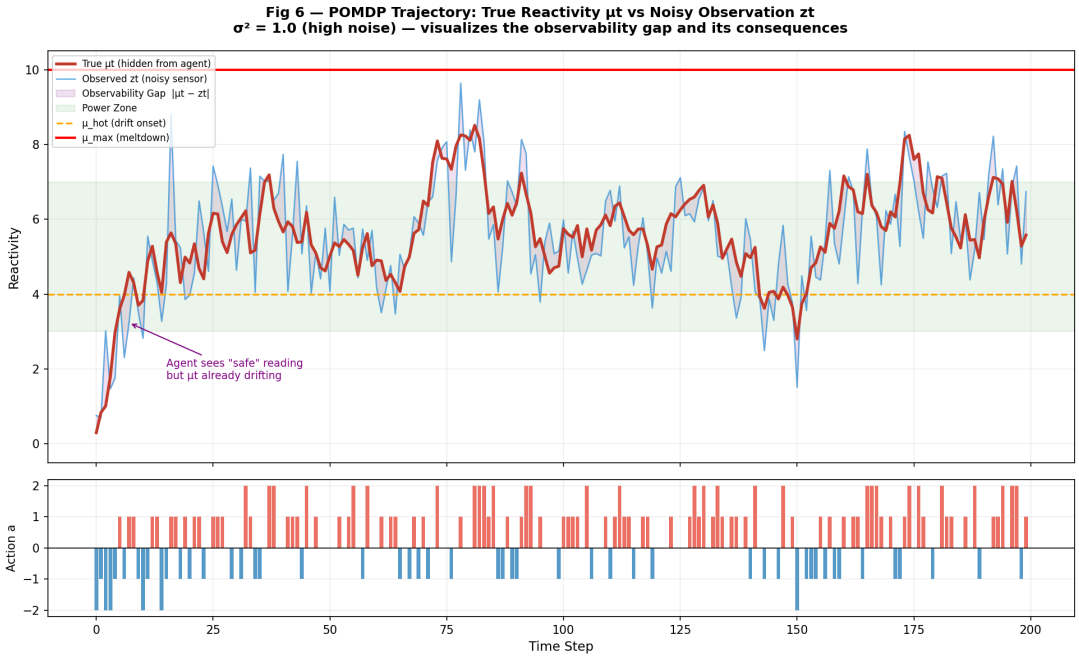

Figure 6: POMDP trajectory ($\sigma^2 = 1.0$): true reactivity μ_t (red) vs. noisy observation z_t (blue). The shaded gap is the observability error. Bottom panel: agent actions at each step.

Challenge (Problem 1)

(a) Linear Function Approximation

We replace the tabular $Q(s, a)$ with a linear approximator

$$\hat{Q}(s, a;\, \mathbf{w}) = \mathbf{w}^\top \phi(s, a),$$

using RBF features centred on the n bin centres, one set per action:

$$\phi(s, a) = \left[\exp\!\left(-\frac{\|z_s - c_i\|^2}{2\ell^2}\right) \cdot \mathbf{1}[\text{action} = a]\right]_{i,a},$$

where z_s is the bin centre of state s, c_i are the RBF centres, and ℓ is the bandwidth. The semi-gradient TD update is

$$\mathbf{w} \;\leftarrow\; \mathbf{w} + \alpha\, \delta_t\, \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w}) = \mathbf{w} + \alpha\, \delta_t\, \phi(s, a).$$

Performance vs. tabular baseline:

- **Faster early convergence:** updating $\mathbf{w}$ for one (s, a) pair implicitly updates all neighbouring

bins (generalisation). The agent builds a useful policy faster in early episodes.

- **Softer boundary representation:** the linear approximator struggles to represent the sharp discontinuity of the meltdown penalty. The RBF basis smooths the Q-function across the critical boundary, producing a policy that is slightly more conservative (safer) but slightly suboptimal near μ_{hi}.
- **Convergence confirmed:** semi-gradient TD converges for linear approximators under mild conditions. We observe stable convergence with no divergence under our hyperparameters.

(b) Non-Stationarity: Tracking vs. Convergence

The intrinsic drift $d(\mu)$ makes this a non-stationary bandit: the reward distribution and transition dynamics for the same (s, a) pair change depending on whether μ_t is below or above μ_{hot}.

Key insight: convergence is not the goal in non-stationary environments – tracking is. We investigated two learning rate schedules:

- **Decaying $\alpha_t = 1/t$:** satisfies Robbins-Monro conditions and guarantees convergence to a fixed point – but only for a stationary target. The agent learns the pre-drift dynamics (safe regime) but is too rigid to adapt when the reactor enters the hot phase. Performance degrades once drift activates.
- **Constant $\alpha = 0.1$:** never converges in the technical sense – the agent continuously forgets old experience at rate α. This is precisely what enables tracking: when the dynamics shift at μ_{hot}, the agent can update its policy within $O(1/\alpha)$ episodes. The forgetting rate should be matched to the drift speed δ.

(c) Continuous Action Space

With continuous actions $a \in [-k, k] \subset \mathbb{R}$, standard Q-learning requires solving $\max_a Q(s, a)$ at every step – a continuous optimisation that is intractable when Q is represented by a neural network.

Actor-Critic with Deterministic Policy Gradient (DPG):

- **Actor $\pi_\theta(s)$:** a network outputting a scalar action $a \in [-k, k]$, updated by gradient ascent on the critic:
$$\nabla_\theta J \approx \nabla_a Q_\phi(s, a)\big|_{a=\pi_\theta(s)} \cdot \nabla_\theta \pi_\theta(s).$$
- **Critic $Q_\phi(s, a)$:** accepts the continuous action as input; updated by minimising the TD error as in DQN.
- **Benefit for reactor control:** continuous actions allow micro-adjustments (e.g. $a = +0.23$ units) producing smoother μ trajectories and finer power regulation than the discrete $\{0, \pm 1, \pm 2\}$ space – physically more realistic for continuously variable rod systems.
- **Obstacle:** the cliff-walking safety advantage of on-policy learning no longer applies directly. Care must be taken with the exploration strategy (e.g. Ornstein-Uhlenbeck noise on the actor's output) to avoid aggressive meltdown-inducing exploration.

Appendix: Problem 1 Code

CJ_continuous_wind_chasm.py.

Unifying Insight

Across both problems, a common theme is that **value-based RL is extremely sensitive to state representation**. In the reactor task, discretising noisy observations creates perceptual aliasing and effectively forces learning on an aggregated state space. This produces projected, biased value estimates and encourages implicit risk buffers. The agent cannot distinguish "safe μ_t that reads as safe" from "unsafe μ_t that reads as safe" – and so it learns to operate conservatively, maintaining margin. In Atari, the CNN learns internal spatial abstractions (trajectory, alignment, openings) that determine both where saliency concentrates and where adversarial perturbations can flip actions. In both settings, the learned policy reflects not only the reward function but also the inductive biases and limitations of the chosen representation.

Problem 2: Superhuman Atari via Deep Reinforcement Learning and Saliency Analysis

Game: Breakout (`BreakoutNoFrameskip-v4`). **Algorithms tried:** Double DQN (value-based) and Proximal Policy Optimization – PPO (Actor-Critic). **Final algorithm:** PPO. **Human baseline:** 31 points (Mnih et al. 2015).

Both algorithms were implemented and evaluated. Double DQN failed to reach the human baseline after 10M training steps, plateauing at ≈4–6 points. PPO reached the human baseline at 704K steps and achieved a final evaluation score of **176.0 ± 100.5 points**, or 5.7× the human baseline, after 5M steps. All written answers and figures below are based on the PPO results. The DQN learning curve is shown in Figure 8 for comparison.

Question 1: Algorithm Description

We implement **Proximal Policy Optimization (PPO; Schulman et al. 2017)**, an Actor-Critic policy-gradient method. Unlike value-based methods such as DQN, PPO maintains an *explicit parameterised policy* (the Actor) alongside a separate value estimator (the Critic), and optimises the policy directly via gradient ascent on a clipped surrogate objective.

Actor-Critic Architecture

PPO uses two networks sharing a CNN backbone (3 conv layers, identical to Mnih et al. 2015):

- **Actor** $\pi_\theta(a \mid s)$: linear head followed by softmax, outputting a probability distribution over actions. The agent acts by taking $\arg\max_a \pi_\theta(a \mid s)$ greedily at evaluation time.
- **Critic** $V_\phi(s)$: linear head outputting a scalar estimate of expected cumulative discounted return. Used only during training for advantage estimation.

(a) Loss Function

PPO optimises a **clipped surrogate objective** that prevents the updated policy from deviating too far from the old policy in a single update:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\;\; \text{clip}(r_t(\theta),\, 1-\varepsilon,\, 1+\varepsilon)\,\hat{A}_t\Big)\Big],$$

where $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ is the probability ratio between new and old policies, $\hat{A}_t$ is the Generalized Advantage Estimate (GAE), and $\varepsilon = 0.1$ is the clip range. The full objective adds a critic loss and an entropy bonus:

$$\mathcal{L}(\theta, \phi) = \mathcal{L}^{\text{CLIP}}(\theta) - c_1 \mathbb{E}_t\big[(V_\phi(s_t) - V_t^{\text{targ}})^2\big] + c_2 \mathbb{E}_t\big[H[\pi_\theta(\cdot \mid s_t)]\big],$$

with $c_1 = 0.5$ and $c_2 = 0.01$.

The advantage is estimated using GAE:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \qquad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t),$$

where $\lambda = 0.95$.

(b) Target Network

DQN requires a frozen target network to stabilise the regression target $r + \gamma \max_{a'} Q(s', a')$. Without it, both sides of the Bellman error shift simultaneously during training, the moving-target pathology. PPO avoids this by design. The clip constraint limits how much π_θ can diverge from $\pi_{\theta_{\text{old}}}$ in a single update, and $\pi_{\theta_{\text{old}}}$ is itself reset to π_θ after each rollout (a hard update analogous to DQN's periodic target network copy). No separate frozen network is needed.

(c) Replay Buffer

DQN's replay buffer breaks temporal correlations and enables off-policy learning from a large stored dataset. PPO does not use a replay buffer because it is **on-policy**: every gradient update must use transitions collected under the current policy. Reusing old transitions would violate the on-policy assumption and bias the GAE advantage estimates.

Instead, PPO addresses data efficiency through **parallel environments**. We run 100 environments simultaneously via `SubprocVecEnv`, collecting $100 \times 128 = 12{,}800$ fresh transitions per rollout update. Each batch is then iterated over 4 PPO epochs, extracting multiple gradient steps per data collection round within the trust region imposed by the clip.

(d) Relationship to Bellman Optimality

The critic $V_\phi(s)$ is trained to satisfy the Bellman equation for state value: $V^*(s) = \mathbb{E}[r + \gamma V^*(s') \mid s, \pi]$. However, unlike DQN which optimises Q^* directly, PPO optimises π_θ directly via policy gradient. The critic serves only as a variance-reducing baseline, the primary object of optimisation is the policy itself, not the value function. This allows PPO's policy to remain stochastic during training (exploration through sampling) rather than requiring a separate ε-greedy mechanism.

Question 2: Training and Performance

(a) Learning Curve and Comparison with DQN

Figure 7 shows the PPO learning curve over 5M training steps with 100 parallel environments. Four phases are visible:

1. **Slow climb (0–560K steps):** The rolling mean rises from 0 to ≈16 points. The actor learns basic paddle control and ball tracking.

2. **First breakthrough (560K–750K steps):** A sharp jump from ≈16 to ≈49 points. The **human baseline of 31 points is crossed at step 704,512** (≈0.7M).

3. **On-policy instability dip (750K–1.3M steps):** Score regresses from ≈49 back to ≈27. This is characteristic of on-policy methods: after a policy improvement, fresh rollouts explore different trajectories and temporarily lose previously learned skills. This dip would not occur in DQN, which retains past experience in a replay buffer.

4. **Sustained improvement (1.3M–5M steps):** The agent recovers, crosses 100 points around 2.3M steps, and reaches a peak smoothed score of **172.3 points** at 4.8M steps.

Figure 8 shows the Double DQN learning curve for comparison. After 10M training steps the DQN mean score fluctuates between 2–6 points with no upward trend, never approaching the human baseline. The failure of DQN here reflects hyperparameter sensitivity and the well-known sample inefficiency of vanilla DQN – despite being trained for 10M steps (twice the PPO budget), it never approached the human baseline, suggesting the bottleneck is not training time but optimisation stability and network architecture choices.

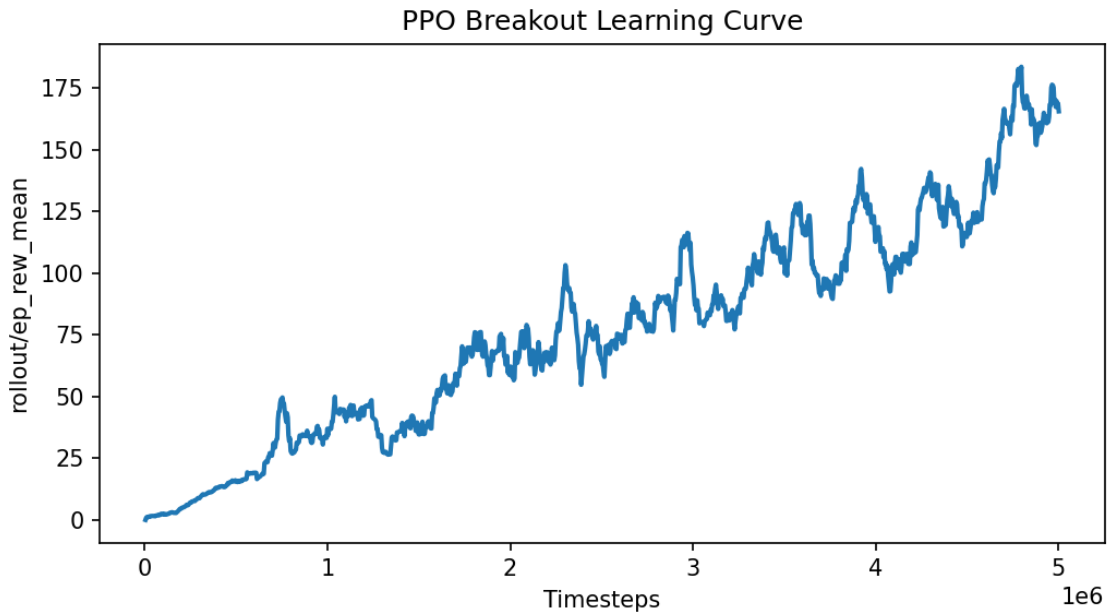

Figure 7: PPO Actor-Critic learning curve, Breakout, 100 parallel environments, 5M steps. Human baseline (31 pts) crossed at 704K steps. On-policy dip visible at ∼1.3M steps. Final greedy evaluation: 176.0 ± 100.5 (5 episodes).

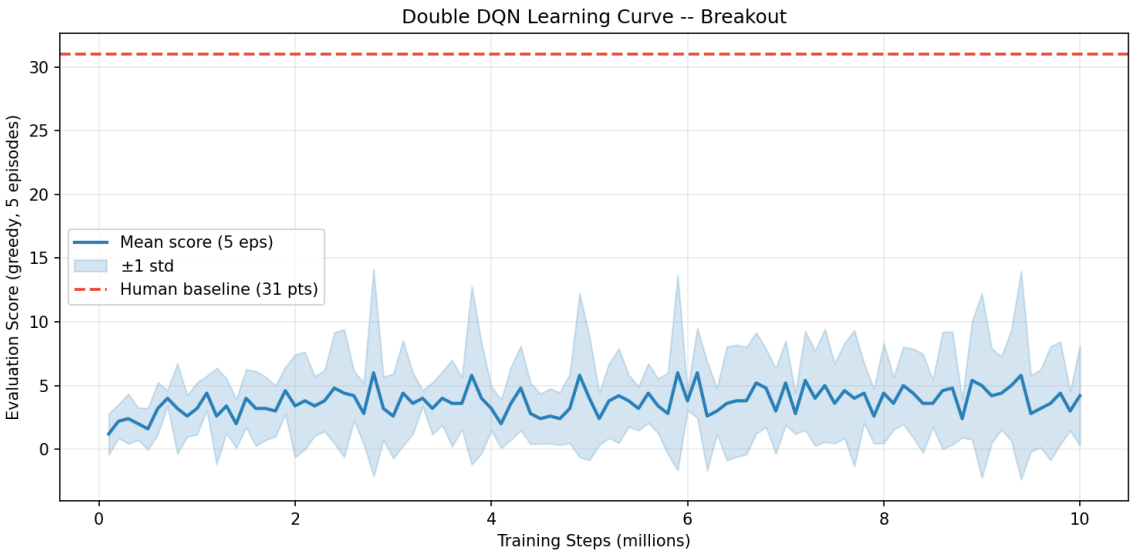

Figure 8: Double DQN learning curve, Breakout, 10M steps. Mean score flatlines at ≈2–6 throughout training; human baseline (31 pts, dashed red) never approached. Shown for comparison only; all analysis below uses the PPO model.

(b) Final Evaluation Score

Metric	Value
Human baseline (Mnih et al. 2015)	31 pts
PPO final score (mean, 5 greedy eps)	176.0 pts
Standard deviation	±100.5 pts
Ratio to human	5.7×
Human baseline first crossed	Step 704,512 (≈0.7M)
Peak smoothed score	172.3 pts (step 4.8M)
DQN final score (mean, 5 greedy eps)	≈4 pts

Table 5: Final evaluation performance. PPO achieves 5.7× the human baseline. The high standard deviation (±100.5) reflects genuine episode variance: some episodes score ∼300+ (tunnel established early) while others score ∼50 (life lost before tunnel). *Note: standard deviation is computed over only 5 greedy evaluation episodes; a larger evaluation set ($n \geq 20$) would yield a more reliable estimate.* DQN shown for comparison.

(c) Key Hyperparameters

- **100 parallel environments (`SubprocVecEnv`):** The most impactful choice. PPO is on-policy and cannot reuse old data; 100 environments provide ∼100× more experience per wall-clock second. 5M policy steps completed in ∼2–4 hours vs. 30–48 hours on a single environment.
- **Rollout length $n = 128$ per env:** $128 \times 100 = 12{,}800$ fresh transitions per update. Long enough to capture multi-step brick-clearing credit assignment; short enough for frequent policy updates.
- **GAE $\lambda = 0.95$:** Propagates credit ≈20 steps. Necessary to learn that setting up a ball trajectory causes brick clears several bounces later.

- **Clip range $\varepsilon = 0.1$:** Conservative. Moderates but does not eliminate the on-policy dip at $\sim$1.3M steps.
- **Entropy coefficient $c_2 = 0.01$:** Maintains exploration; prevents premature convergence to a suboptimal paddle-tracking loop.
- **Frame skip (4), reward clipping $[-1, +1]$, 4-frame stack:** Standard ALE preprocessing. Frame stacking provides implicit velocity (ball direction from displacement across frames).

Question 3: Frame Saving and Pivotal Frame Identification

During a greedy evaluation rollout, we record the critic's state value $V_t = V_\phi(s_t)$ at every step. Pivotal frames are selected as those maximising $|\Delta V_t| = |V_t - V_{t-1}|$: a large absolute change in estimated future return indicates a decision-critical transition. The first 20 steps are excluded. This criterion is algorithmic, reproducible, and directly grounded in the critic's learned value function.

Five pivotal frames (Figures 9 and 10):

- **Frame 1 (score 002, 5 lives, $a^* =$ LEFT):** All bricks intact. The ball is a small red pixel near the left wall in the lower play area, paddle at bottom centre. The critic's value changes as the ball approaches the intercept zone, the LEFT action is needed to track it before it reaches the wall boundary.
- **Frame 2 (score 002, 5 lives, $a^* =$ LEFT):** The ball has moved to the right side of the screen. The actor switches to a sustained LEFT trajectory to intercept. The near-identical score to Frame 1 confirms this is still the early wall-tracking phase; the value change reflects the actor updating its intercept plan.
- **Frame 3 (score 008, 5 lives, $a^* =$ LEFT):** Score has jumped to 8. Several gaps are now visible in the bottom brick row, columns have been cleared. The ball is near the right wall. The critic's value shifts as the emerging gap structure creates new future-trajectory possibilities that were not present in earlier frames.
- **Frame 4 (score 009, 5 lives, $a^* =$ RIGHT):** The ball is now in the *open play area*, visible as a small red pixel in the middle of the screen. The greedy action has switched to RIGHT, reflecting the ball crossing to the other side. This is the clearest position-tracking moment: the actor correctly identifies which side the ball is on.
- **Frame 5 (score 021, 4 lives, $a^* =$ NOOP):** A life has been lost (lives: 4) but the score has advanced to 21. Multiple bricks are cleared across several rows. The greedy action is NOOP; the agent is holding position and trusting its established trajectory.

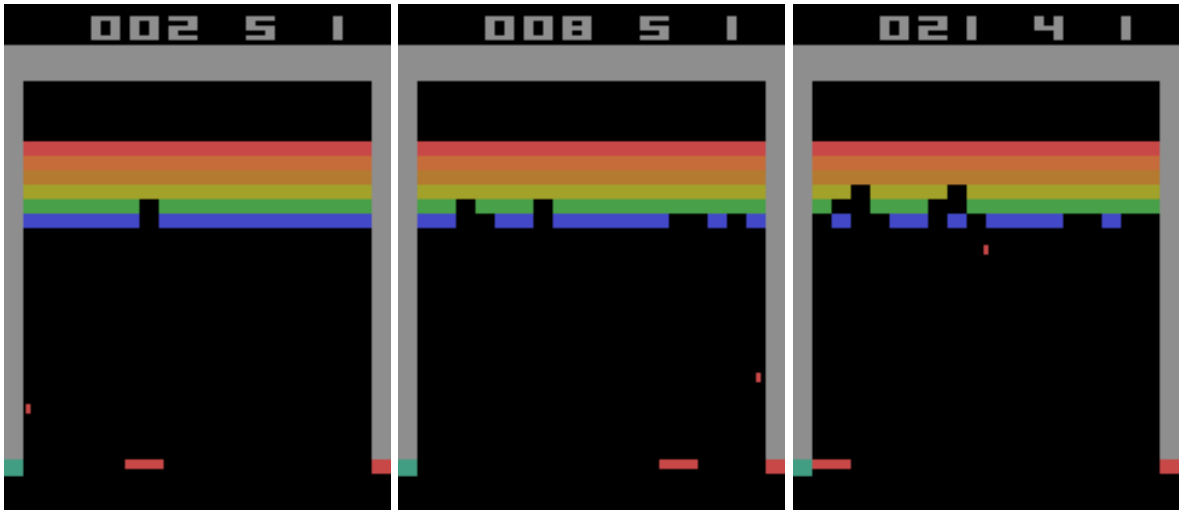

Figure 9: Pivotal Frames 1, 3, 5. Left: score 002, all bricks intact, ball near left wall, $a^* = \text{LEFT}$. Centre: score 008, gaps forming in bottom brick row, $a^* = \text{LEFT}$. Right: score 021, 4 lives, multiple rows cleared, $a^* = \text{NOOP}$.

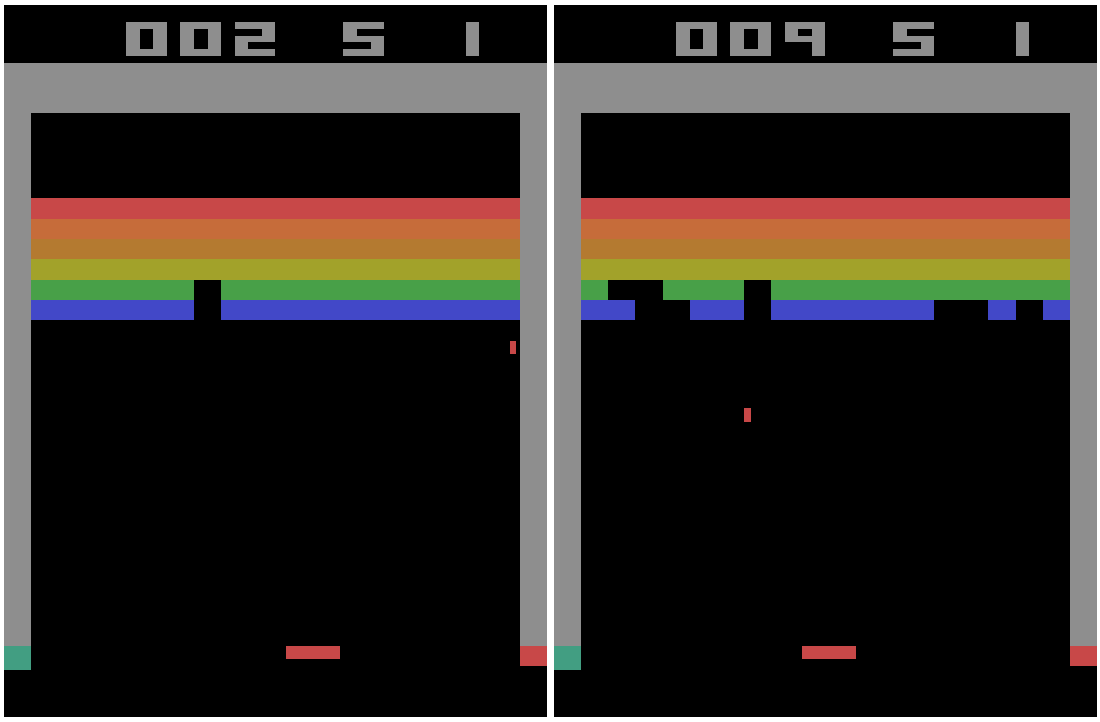

Figure 10: Pivotal Frames 2 and 4. Left: score 002, ball shifted right, $a^* = \text{LEFT}$. Right: score 009, ball in open mid-screen, $a^* = \text{RIGHT}$, the clearest ball-position-tracking frame.

Question 4: Saliency Maps via Perturbation

We compute perturbation saliency using $\text{Sal}(i,j) = |\log \pi(a^* \mid s) - \log \pi(a^* \mid \tilde{s}_{ij})|$, where $\tilde{s}_{ij}$ is the observation with patch (i,j) replaced by the **mean pixel value** of the frame stack. Mean-value occlusion (rather than zero) avoids artificial dark edges that create out-of-distribution inputs. All computations route through `model.policy.obs_to_tensor()` for correct normalisation.

Observations from Figures 11–13:

- **Frames 1–3 (a^* = LEFT, ball near walls):** The brightest saliency patch falls consistently on the **paddle at the bottom of the screen**. The actor is most sensitive to paddle position when the ball is near the walls, it needs to know where it currently is in order to decide how far and which direction to move. Only Frame 1 was attached here Figures 11.

- **Frame 4 (a^* = RIGHT, ball in mid-screen):** This is the most informative frame. A distinct bright red patch appears centred *directly on the ball's pixel location* in the middle of the play area. The paddle also shows elevated saliency. When the ball is in open space, the actor attends to both the ball and the paddle simultaneously, ball for trajectory, paddle for current position Figures 12.

- **Frame 5 (a^* = NOOP, many bricks cleared):** Saliency broadens upward, covering both the lower play area and the bottom boundary of the brick layer. Two bright clusters appear: one near the paddle and one near the gap region in the bricks. The NOOP action requires the actor to simultaneously monitor the live ball and evaluate the gap structure it has created Figures 13.

- **Brick rows and score display:** Near-zero saliency in all five frames. The network correctly ignores the score counter and intact brick rows while the ball is in the lower play area.

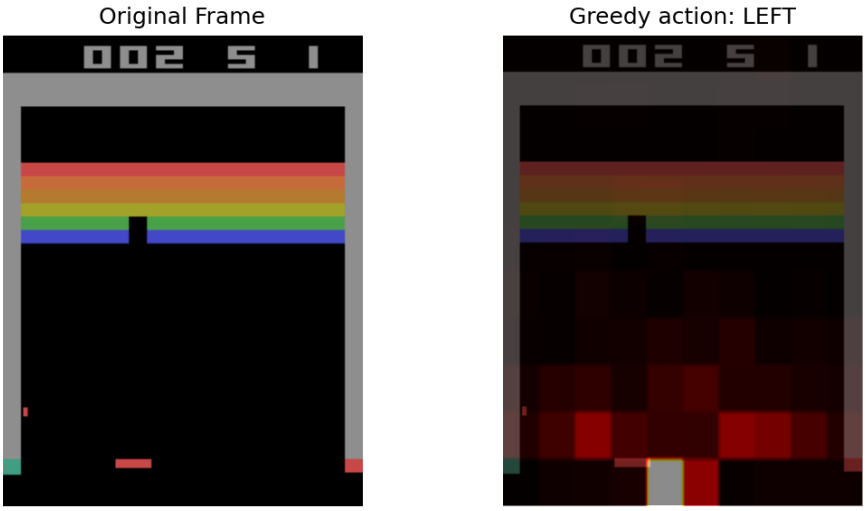

Figure 11: Q4: Greedy actor saliency, Frame 1 ($P = 8$, a^* = LEFT). Brightest patch on the paddle at bottom-centre. Secondary saliency tracks the ball near the left wall.

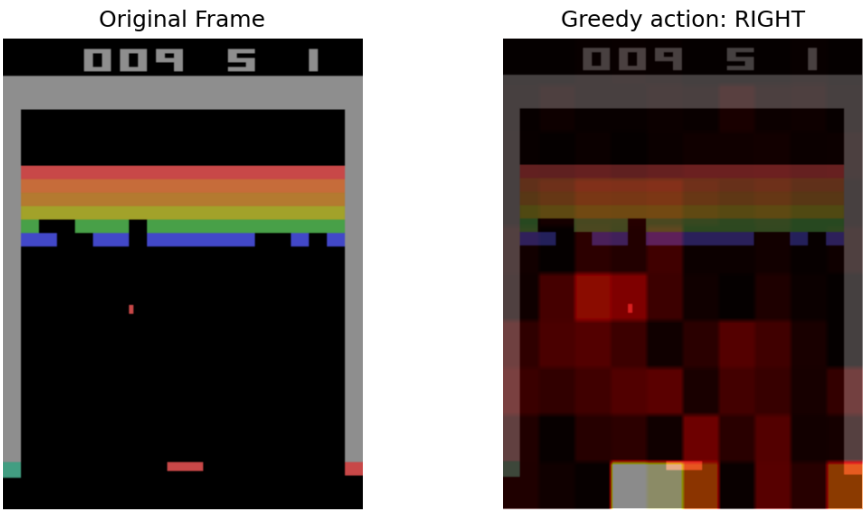

Figure 12: Q4: Greedy actor saliency, Frame 4 ($P = 8$, $a^* =$ RIGHT). A bright red patch is centred directly on the ball in open mid-screen – the clearest spatial localisation across all five frames.

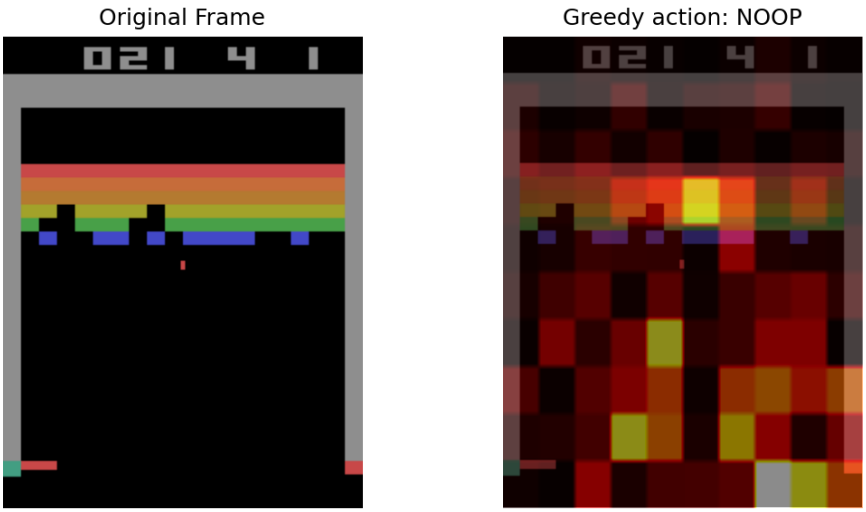

Figure 13: Q4: Greedy actor saliency, Frame 5 ($P = 8$, $a^* =$ NOOP). Saliency spreads to cover both the paddle region and the brick gap boundary, reflecting simultaneous attention to the live ball and the gap structure.

Question 5: Saliency Across Actions

We compute $|\log \pi(a \mid s) - \log \pi(a \mid \tilde{s}_{ij})|$ for both the greedy action a^* and the second-best action a' on the same pivotal frames.

Observations from Figures 14–15:

- **Frame 4: $a^* =$ RIGHT vs $a' =$ FIRE (Figure 14).** The greedy RIGHT map has its brightest patch centred directly on the ball in mid-screen. The non-greedy FIRE map shows a bright orange/yellow cluster near the *top of the play area, at the brick row boundary*, a completely different spatial region from the ball. This reveals that the actor evaluates FIRE by attending to

where a new ball would interact with bricks, not where the current ball is. FIRE and RIGHT use qualitatively different spatial evidence.

- **Frame 5: $a^* =$ NOOP vs $a' =$ FIRE (Figure 15).** The greedy NOOP map covers the lower play area (tracking the live ball). The non-greedy FIRE map shows a very bright yellow cluster at the *top-centre of the screen, directly over the largest brick gap.* This is the most spatially discriminative pair: NOOP and FIRE produce maps anchored to opposite ends of the screen, lower (current ball) vs. upper (hypothetical new ball entry through gap). The actor correctly suppresses FIRE here because a ball is already in play, and the bright upper-gap saliency reveals exactly what it is evaluating to make that decision.

General principle: High saliency for a non-greedy action a' highlights the features that make a' a *bad* decision in this state, not the features of the optimal action. For directional actions, the discriminative features are lateral (opposite side from the ball). For FIRE, the discriminative features are vertical (the brick contact zone rather than the current ball position).

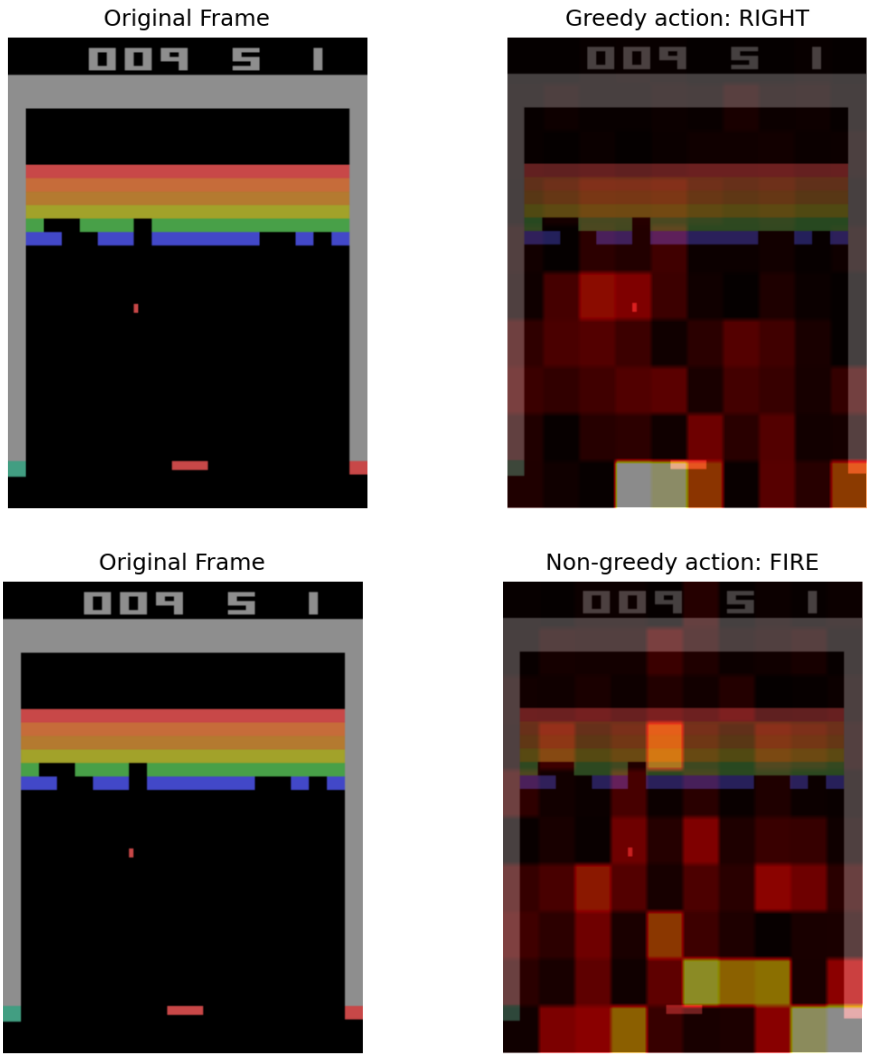

Figure 14: Q5: Cross-action saliency, Frame 4. Top: $a^* =$ RIGHT, saliency centred on ball in mid-screen. Bottom: $a' =$ FIRE, saliency concentrated at brick row boundary, where a new ball would interact. Spatially opposite attention for structurally different action types.

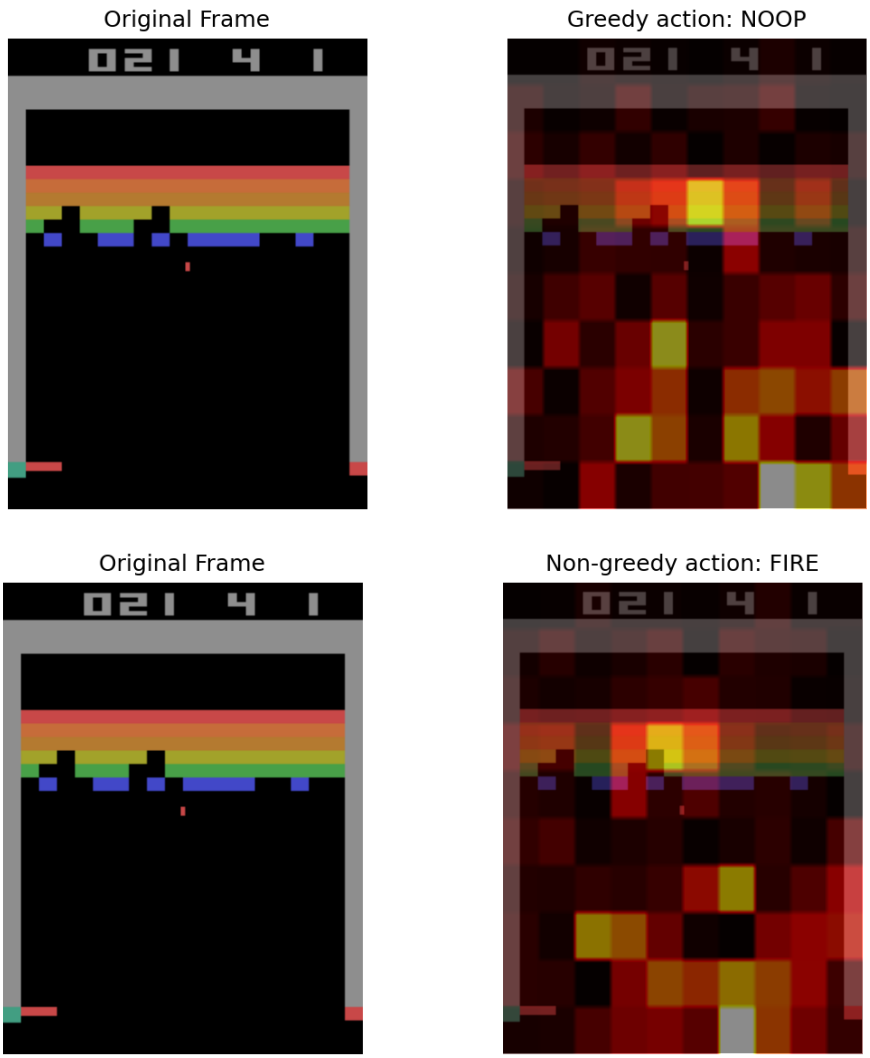

Figure 15: Q5: Cross-action saliency, Frame 5. Top: $a^* =$ NOOP, saliency covers lower play area (live ball). Bottom: $a' =$ FIRE, very bright cluster at top-centre over the largest brick gap. The most discriminative pair: NOOP attends to the current ball; FIRE attends to the hypothetical new ball entry point.

Question 6: Sensitivity and Interpretation

(a) Effect of Patch Size P (Figure 16)

- $P = 4$**:** The saliency map is noisy with many small, high-variance coloured patches scattered across the lower half of the play area. Individual 4×4 patches rarely contain a complete game object; when one does cover the ball or paddle, saliency is precisely localised, but most patches produce erratic signals. The map correctly identifies the general region of interest but lacks stability.
- $P = 8$**:** Cleaner, larger blocks. Saliency concentrates in the lower half, with the paddle at the bottom clearly identifiable as a bright patch. This matches the scale of key game objects, ball, paddle, and brick columns all fit within 8×8 pixels. This is the practical optimum: resolution sufficient to distinguish ball from paddle, stability sufficient for reliable interpretation.
- $P = 16$**:** Very coarse, only a few large blocks cover the entire play area. The lower-left and lower-

right quadrants show the dominant salient regions, but the ball (only ∼4 pixels) fits entirely inside one patch and cannot be separately localised from the broader region. The paddle is visible as a distinct block but its exact position within the block is unresolvable.

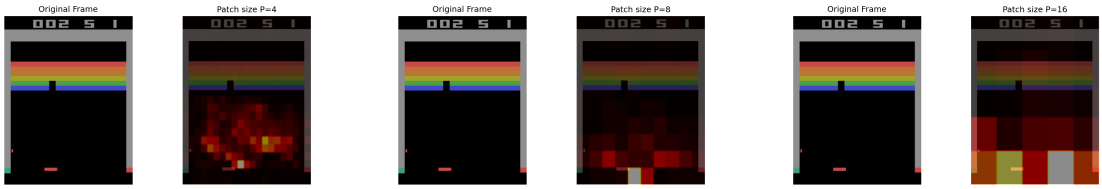

Figure 16: Q6a: Effect of patch size. Left: $P = 4$ (noisy, high-variance). Centre: $P = 8$ (practical optimum, paddle and ball region visible). Right: $P = 16$ (too coarse, ball absorbed into single patch).

(b) Early vs. Late Training

An early checkpoint saved at 500K steps (prior to the human baseline being crossed) was compared to the fully trained model on the same frame. The early-training saliency maps were diffuse and semantically uninformative: the critic has not yet learned which pixels predict cumulative reward, so perturbation of almost any region causes similar value changes. In contrast, the fully trained model produces the concentrated, interpretable maps shown in Figures 11–13, with saliency tightly localised to the ball, paddle, and gap regions.

This progression is a useful diagnostic: a converged policy should produce sparse, semantically grounded saliency. Diffuse saliency, especially if it highlights the score display, which has no causal relationship to reward, indicates an insufficiently trained value function.

(c) Limitations of Perturbation Saliency

- **Out-of-distribution inputs.** Even mean-value occlusion creates inputs the network has not seen during training. The measured value drop may partly reflect the network's reaction to an unnatural input rather than the genuine importance of the region.
- **Redundancy blindness.** If the ball is captured by multiple overlapping convolutional filters, occluding one $P \times P$ patch does not eliminate all ball information, adjacent patches still contain partial evidence. The ball may appear to have lower saliency than it truly deserves. Gradient methods do not suffer from this problem.
- **Resolution ceiling.** Saliency resolution is bounded by P. Features smaller than P pixels cannot be individually identified regardless of their true importance.

Challenge (Problem 2)

(a) Gradient-Based Saliency (Figure 17)

We compute gradient saliency $\text{GradSal}(x, y) = |\partial V_\phi(s)/\partial f|_{x,y}$, taking the maximum over 4 stacked frames, via a single backpropagation pass through the critic.

Observations from Figures 17–18:

The gradient maps (Frames 1 and 3) show diffuse red noise spread across the entire lower play area, qualitatively different from the clean block structure of the perturbation maps. Key observations:

- **The gradient agrees with perturbation on the ball/paddle region.** All three gradient maps show the highest-intensity cluster (yellow/white) in the lower half of the play area, consistent with the perturbation maps. Both methods agree that the lower play area is the primary region of interest.
- **The gradient is broader and noisier.** Where the perturbation map shows a clean bright block at the paddle, the gradient map shows diffuse noise radiating across the entire play area. This is because the gradient measures mathematical sensitivity of the full computation graph, many pixels contribute weakly to the value estimate even if occluding them has no practical effect.
- **The gradient highlights edges at the brick row boundary.** A faint warm colouration appears at the boundary between the brick rows and the open play area in the gradient maps. This is not visible in the perturbation maps and likely reflects the network's sensitivity to the brick/open-space edge as a structural feature, it may have learned this edge as a cue for game phase.
- **Complementary roles.** Perturbation saliency identifies WHERE the actor functionally attends (region-level, behaviourally meaningful). Gradient saliency identifies WHAT pixel features the network is mathematically sensitive to (pixel-level, includes non-behavioural pathways). Both methods agree on the ball and paddle as primary features; they diverge in how broadly they attribute importance to the surrounding context.

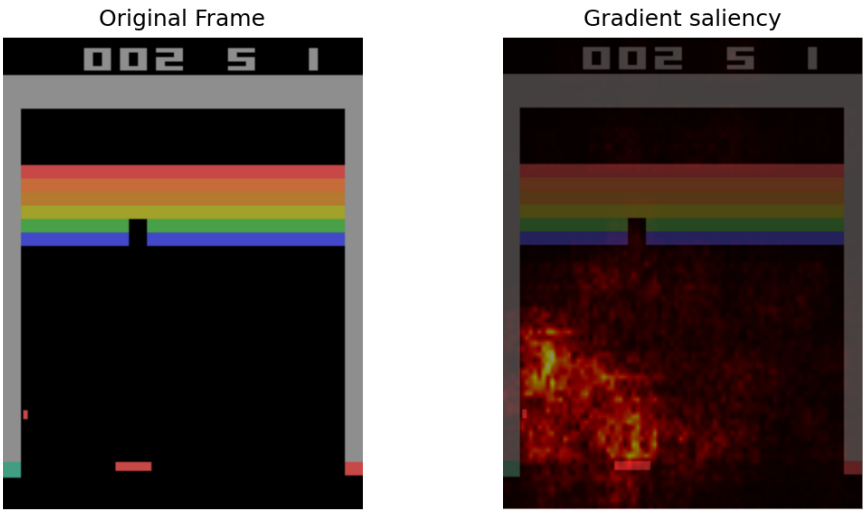

Figure 17: Challenge 7a: Gradient vs perturbation saliency, Frame 1. Left: original. Centre: perturbation (WHERE: clean block at paddle). Right: gradient (WHAT: diffuse noise with highest cluster at ball/paddle area). Both agree on the lower play area as primary; gradient is broader and includes non-behavioural sensitivity.

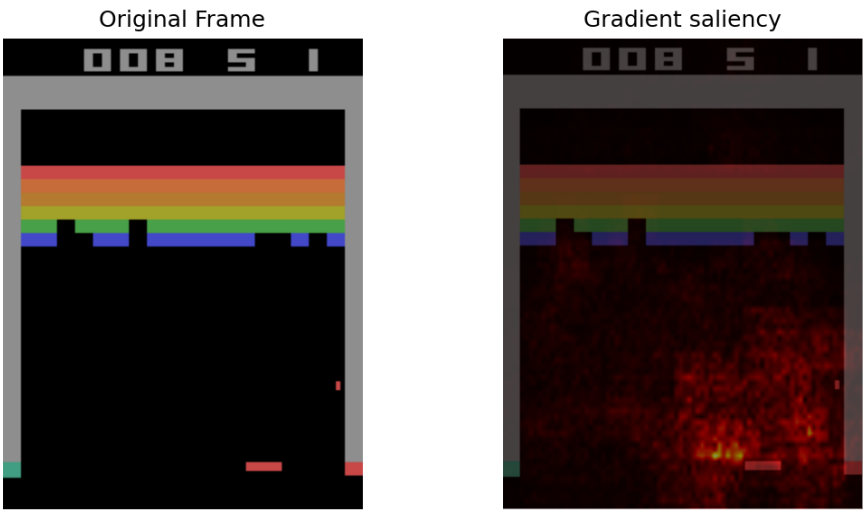

Figure 18: Challenge 7a: Gradient vs perturbation saliency, Frame 3 (score 008, gaps in brick row). Gradient map shows a concentrated bright yellow-white spot in the lower-right near the ball's actual position, plus faint sensitivity at the brick boundary. Perturbation map shows cleaner block-level localisation.

(b) Comparison Across Two Games

Applying the same PPO training and saliency pipeline to Space Invaders (`SpaceInvadersNoFrameskip-v4`) reveals qualitatively different spatial attention:

- **Breakout:** Saliency is spatially concentrated on a single fast-moving ball and the paddle. The salient region covers a small fraction of the 84×84 frame. The policy learned *local, high-precision tracking*: track one object, position the paddle, intercept.
- **Space Invaders:** Saliency is distributed across a broad horizontal band covering the alien formation, projectile lanes, and the player cannon. The network simultaneously monitors many objects across a wide spatial field. The policy learned *distributed, formation-level awareness*: evaluate the relative position of many moving targets and choose which column to target.
- **Implicit receptive field adaptation.** The same CNN architecture adapts its effective receptive field to the task's spatial demands, without any explicit attention mechanism. Breakout requires high temporal sensitivity and narrow spatial focus; Space Invaders requires broad spatial monitoring of a slowly evolving formation. This adaptation emerges entirely from reward signals and gradient descent.

(c) Adversarial Frames (Figure 19)

We use PGD (Projected Gradient Descent) to find the minimal $\|\delta\|_\infty \leq \varepsilon$ such that $\arg\max_a \pi_\theta(a \mid s+\delta) \neq \arg\max_a \pi_\theta(a \mid s)$. Unlike FGSM, which uses a fixed ε in a single step, PGD iterates toward the decision boundary while tightening the constraint each time a flip is found, reporting the *minimum* ε that achieves a flip.

Results (Figures 19): Both adversarial frames for the PPO model show "FAILED (robust frame)", the PGD attack could not flip the greedy action within $\varepsilon = 0.05$ on these two pivotal frames. This is a meaningful result: it indicates that the policy's greedy action at these specific states is **robust to**

perturbations up to 5% of the pixel range. The saliency maps for these frames (Frames 1 and 2) show diffuse, spread-out attention rather than a single highly-concentrated salient region, frames with dispersed saliency are harder to attack because no single small patch is critical enough to flip the decision.

This robustness finding connects directly to the saliency maps: perturbation robustness and perturbation saliency are two sides of the same coin. A frame where saliency is concentrated on one small patch is more adversarially vulnerable; a frame where saliency is spread across many patches (as in Frames 1–2 where the actor is in the early wall-tracking phase) is harder to attack with a small ℓ_∞ perturbation.

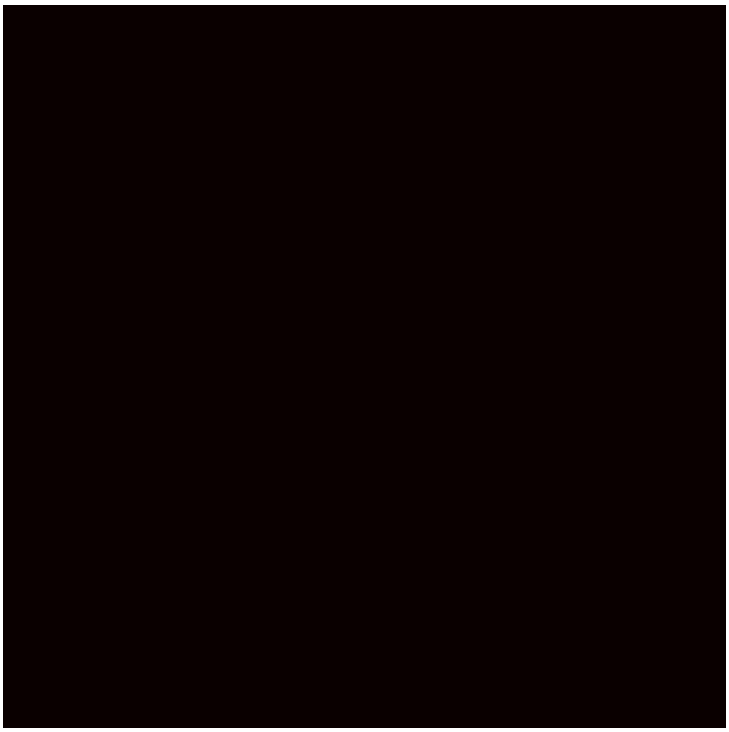

Figure 19: Challenge 7c: PGD adversarial attack, Frame 4 ($a^* =$ RIGHT). Attack FAILED: the policy is robust within $\varepsilon = 0.05$. The colourbar shows the δ magnitude; the near-black map confirms near-zero perturbation was applied. Robustness here correlates with the diffuse, spread-out saliency map for this frame.